

MANUEL MASTERCLASS

ÉDITION EXÉCUTIVE 2026

DÉVELOPPEMENT WEB & MOBILE DE A À Z

Guide complet d'ingénierie logicielle : de la rédaction du Cahier des Charges (CdC) et des Termes de Référence (TDR) jusqu'à la conception d'APIs sécurisées, au développement Web & Mobile, à la containerisation Docker, à la vente commerciale et à la conviction client.

AUTEUR & RÉALISATION

Salomon Tchibkere

EXPERTISE TECHNIQUE

Développeur Full-Stack & Consultant IT

LANGAGES & FRAMEWORKS

React, React Native, Node.js, TS

Réalisé par Salomon — Développeur Full-Stack & Consultant IT

Tous droits réservés © 2026

Sommaire Détaillé du Cours

Module 0 : Rédaction du Cahier des Charges & des Termes de Référence (TDR) avec le Client

0.1 Différence entre Cahier des Charges (CdC) et Termes de Référence (TDR)	Page 3
0.2 Structure Type d'un Cahier des Charges Fonctionnel & Technique	Page 4
0.3 Structure Type des Termes de Référence (TDR) & Échéancier Financier	Page 5
0.4 Méthodologie d'Atelier de Cadrage (Discovery Workshop) avec le Client	Page 6

Module 1 : Analyse des Besoins, UX/UI & Architecture Système

1.1 Analyse comparative des architectures (PWA vs Natif vs React Native)	Page 3
1.2 Spécification des exigences & Diagrammes UML / MERISE	Page 4
1.3 Structuration de l'Arborescence Projet Professionnelle	Page 5

Module 2 : Backend API RESTful, Base de Données & Sécurité (Node.js/Express)

2.1 Modélisation avancée de données avec MongoDB Mongoose & PostgreSQL	Page 6
2.2 Développement complet des Contrôleurs & Gestionnaire d'erreurs	Page 8
2.3 Authentification JWT & Middleware d'autorisation basé sur les Rôles (RBAC)	Page 10
2.4 Validation rigoureuse des entrées utilisateurs avec Zod	Page 12

Module 3 : Développement Frontend Web Moderne (React.js & TypeScript)

Module 4 : Développement Mobile Cross-Platform (React Native & Expo CLI)

4.1 Prise en main de React Native : Composants Natifs vs HTML Page 18

4.2 Configuration du Routeur Mobile (React Navigation 6 : Stack + Tabs) Page 19

4.3 Stockage Sécurisé des Tokens (Expo SecureStore) & Capteurs Matériels Page 21

Module 5 : Communication Temps Réel & Sécurité Avancée

5.1 Intégration de WebSockets (Socket.io) pour la messagerie instantanée Page 23

5.2 Sécurisation contre les attaques OWASP Top 10 (Helmet, Rate Limit, Sanitization) Page 25

Module 6 : Containerisation Docker, CI/CD & Déploiement Cloud

6.1 Containerisation de l'API avec Docker & Docker-Compose Page 26

6.2 Pipeline CI/CD GitHub Actions automatisé Page 28

6.3 Génération des Builds Mobiles APK (Android) & IPA (iOS) via Expo EAS Page 29

Module 7 : Guide de l'Autodidacte : Devenir Développeur Full-Stack Autonome

7.1 La Feuille de Route d'Apprentissage Autonome (Roadmap Step-by-Step) Page 30

7.2 Les 5 Erreurs Fatales à Éviter & La Méthode Anti-Tutorial Hell Page 31

Module 8 : Kit Commercial & Négociation Client : Gagner des Projets IT

8.1 Les 7 Questions & Réponses Clés en Entretien Négociation Client Page 32

Réalisé par Salomon — Développeur Full-Stack & Consultant IT

Édition Officielle 2026

9.1 Les 5 Secrets Technologiques de l'Ingénierie Logicielle Pro	Page 36
9.2 Le Secret pour Obtenir des Clients & un Emploi sans Expérience	Page 38
9.3 Réussir les Entretiens Techniques & Négociation du Tarif (TJM)	Page 40

Module 10 : Psychologie de Vente & Conviction Express : Gagner la Confiance du Client en 5 Min

10.1 La Règle des 3 Premières Minutes (Écoute Active & Inversion de Posture)	Page 42
10.2 Le Script Secret en 4 Étapes pour Supprimer tout Doute Client	Page 43
10.3 Les 5 Signaux Comportementaux qui Déclenchent la Confiance Immédiate	Page 44

Module 11 : Le Lexique & Glossaire Exhaustif du Développeur Full-Stack (Plus de 60 Termes)

11.1 Dictionnaire Web, Frontend, UI/UX & React	Page 45
11.2 Dictionnaire Backend, Base de Données & Sécurité (API REST, JWT, RBAC)	Page 46
11.3 Dictionnaire Mobile (React Native, Expo, EAS) & DevOps / Cloud (Docker, CI/CD)	Page 47
11.4 Dictionnaire Business, Gestion de Projet Agile & Freelance (MVP, TJM, TDR)	Page 48

Module 12 : Intégration des API de Paiement (Mobile Money & Stripe)

12.1 Flux de Paiement Mobile Money (Orange Money, MTN, Wave) & Webhooks	Page 49
12.2 Intégration Carte Bancaire avec Stripe API	Page 50

Module 13 : Maintenance, Monitoring & Plan de Reprise d'Activité (PRA)

MANUEL MASTERCLASS — DÉVELOPPEMENT WEB & MOBILE DE A À Z

SALOMON TCHIBKERE

Module 14 : Modèles de Contrats Juridiques & Accords de Confidentialité (NDA)

14.1 Modèle de Contrat de Prestation de Services Informatiques Page 53

14.2 Accord de Non-Divulcation / Confidentialité (NDA) Prêt à Signer Page 55

Méthodologie de Rédaction du Cahier des Charges & des TDR avec le Client

0.1 Différence entre Cahier des Charges (CdC) et Termes de Référence (TDR)

Dans l'ingénierie logicielle et le conseil IT, la réussite d'un projet dépend de la clarté des documents contractuels élaborés avec le client avant tout développement :

Document	Public Cible & Rôle	Objectif Principal	Contenu Majeur
Cahier des Charges (CdC)	Équipe technique, Développeurs & Chef de projet	Définir CE QUE l'application doit réaliser sur le plan fonctionnel et technique.	User Stories, Maquettes UI/UX, Schéma de base de données, Endpoints API, Sécurité & Performance.
Termes de Référence (TDR)	Comité de direction, Bailleurs de fonds, Juristes & Achats	Définir DANS QUEL CADRE JURIDIQUE ET FINANCIER la mission sera exécutée.	Objectifs stratégiques, Calendrier d'exécution, Livrables contractuels, Échéancier de paiement & Propriété intellectuelle.

0.2 Structure Type d'un Cahier des Charges Fonctionnel & Technique

Un Cahier des Charges professionnel rédigé avec le client doit impérativement respecter la structure canonique suivante :

1. **Présentation du Projet & Contexte** : Historique de l'entreprise/ONG, problème à résoudre, vision du
2. **Specifications Fonctionnelles Détaillées** .

o *Rôles Utilisateurs (RBAC)* : Administrateur, Formateur, Étudiant, Visiteur anonyme

Réalisé par Salomon — Développeur Full-Stack & Consultant IT

Édition Officielle 2026

[Bénéfice] .

- o *Parcours Utilisateur & Maquettes Wireframe* : Description écran par écran (Écran de Connexion, Tableau de Bord, Profil).

3. Spécifications Techniques :

- o Technologies préconisées (Ex: React + React Native + Node.js).
- o Hébergement & Disponibilité exigée (SLA > 99.9%).
- o Temps de réponse maximal par requête (< 500ms).
- o Conformité réglementaire (Protection des données personnelles / RGPD).

4. **Modalités de Recette & Jalons de Livraison** : Critères stricts permettant au client de valider et signer la réception de chaque version.

0.3 Structure Type des Termes de Référence (TDR) & Échéancier Financier

Les TDR encadrent la relation contractuelle, les responsabilités juridiques et la planification financière :

ÉCHÉANCIER FINANCIER RECOMMANDÉ PAR JALONS (MILESTONES) :

- **Acompte initial (30%)** : À la signature du contrat et validation du Cahier des Charges & Maquettes Figma.
- **Deuxième Jalon (40%)** : À la livraison et démonstration de la version Beta fonctionnelle (Web & Mobile).
- **Solde Final (30%)** : À la recette finale, mise en production Cloud et transfert du code source complet sur le dépôt GitHub du client.

0.4 Méthodologie d'Atelier de Cadrage (Discovery Workshop)

Pour extraire les vrais besoins du client lors du premier entretien de cadrage, le consultant doit poser

MANUEL MASTERCLASS — DÉVELOPPEMENT WEB & MOBILE DE A À Z

SALOMON TCHIBKERE

• **1. Problème métier** : Quel est le problème métier que cette application doit résoudre immédiatement ?

• 2 Profil des Utilisateurs : Qui utilisera l'application (âge, niveau informatique, environnement de

Réalisé par Salomon — Développeur Full-Stack & Consultant IT

Édition Officielle 2026

- **3. Périmètre MVP (Minimum Viable Product) :** Quelles sont les 3 fonctionnalités indispensables sans lesquelles l'application n'a aucun sens ?
- **4. Intégrations Existantes :** L'application doit-elle communiquer avec des bases de données ou logiciels déjà en place (ERP, CRM, API tierce) ?
- **5. Budget & Contrainte Temporelle :** Quelle est la date limite absolue de livraison et le budget alloué ?
- **6. Indicateurs de Succès (KPIs) :** Comment mesurerez-vous que le projet est un succès dans 6 mois (nombre d'utilisateurs, temps gagné) ?
- **7. Gouvernance & Décision :** Qui est la personne physique unique au sein de votre organisation habilitée à valider et signer les livrables ?

Analyse des Besoins, UX/UI & Architecture Système

1.1 Comparatif Approfondi des Architectures Mobile & Web

Le choix d'architecture conditionne l'évolutivité, le coût financier et la maintenabilité d'une application sur le long terme. Voici l'analyse comparative détaillée :

Critère	Web Application (PWA / React)	Natif (Swift / Kotlin)	Cross-Platform (React Native)
Langages	HTML5, CSS3, JavaScript / TS	Swift (iOS), Kotlin (Android)	JavaScript / TypeScript
Réutilisation de code	100% (sur navigateurs)	0% (2 bases de code séparées)	85% à 95% entre iOS et Android
Déploiement	Instantané (Serveur Web)	Validation App Store / Play Store	Validation Store + OTA Update instantané
Accès Matériel	Restreint (Caméra basique, GPS)	Accès 100% illimité et natif	Accès complet via Bridges Natifs

1.2 Modélisation UML & Arborescence Projet Professionnelle

Une arborescence bien organisée permet d'isoler la couche de données, les routes et la présentation frontend. Voici la structure de dossier recommandée pour un projet d'entreprise :

mon-projet-fullstack/		
	backend/	# API REST Node.js & Express
MANUEL MASTERCLASS — DÉVELOPPEMENT WEB & MOBILE DE A À Z		
SALOMON TCHIBKERE		
	controllers/	# Logique métier (Auth, Projets, Users)

— middlewares/		# Auth JWT. RBAC. Validation Zod
Réalisé par Salomon — Développeur Full-Stack & Consultant IT		Édition Officielle 2026
— services/		# Services externes (Email, Paiement Stripe)
— app.ts		# Point d'entrée de l'application
— .env.example		
— Dockerfile		
— package.json		
— frontend-web/		# Application Web React + Vite
— src/		
— components/		# Boutons, Cards, Modals réutilisables
— context/		# AuthContext, ThemeContext
— hooks/		# Custom Hooks (useAuth, useFetch)
— pages/		# Pages (Home, Login, Dashboard)
— services/		# Client Axios interfacé avec l'API
— mobile-app/		# Application Mobile React Native (Expo)
— src/		
— navigation/		# Stack & Tab Navigators
— screens/		# Écrans mobiles (LoginScreen, HomeScreen)
— services/		# Services partagés et SecureStore

Backend API RESTful, Base de Données & Sécurité

2.1 Connexion à la Base de Données & Schéma Utilisateur

Nous utilisons **Mongoose** (MongoDB) avec un schéma d'utilisateur intégrant le hachage sécurisé du mot de passe avec **Bcrypt**.

```
import mongoose, { Schema, Document } from 'mongoose';
import bcrypt from 'bcryptjs';

export interface IUser extends Document {
  nom: string;
  email: string;
  motDePasse: string;
  role: 'ETUDIANT' | 'FORMATEUR' | 'ADMIN';
  comparerMotDePasse(motDePasseSaisi: string): Promise<boolean>;
}

const UserSchema: Schema = new Schema({
  nom: { type: String, required: true, trim: true },
  email: { type: String, required: true, unique: true, lowercase: true },
  motDePasse: { type: String, required: true, minlength: 6 },
  role: { type: String, enum: ['ETUDIANT', 'FORMATEUR', 'ADMIN'], default: 'ETUDIANT' },
  { timestamps: true }
});

// Hachage automatique avant sauvegarde
UserSchema.pre<IUser>('save', async function(next) {
  if (!this.isModified('motDePasse')) return next();
  const salt = await bcrypt.genSalt(10);
  this.motDePasse = await bcrypt.hash(this.motDePasse, salt);
  next();
});

// Méthode d'instance pour vérifier le mot de passe
UserSchema.methods.comparerMotDePasse = async function(motDePasseSaisi: string): Promise<boolean> {
```

2.2 Middleware d'Authentification JWT & Contrôle d'Accès (RBAC)

Le middleware suivant extrait le token JWT envoyé dans l'en-tête HTTP `Authorization: Bearer <TOKEN>` et vérifie les droits d'accès :

```
import { Request, Response, NextFunction } from 'express';
import jwt from 'jsonwebtoken';

export interface AuthRequest extends Request {
  user?: { id: string; role: string };
}

export const authentifierJWT = (req: AuthRequest, res: Response, next: NextFunction) => {
  const authHeader = req.headers.authorization;
  if (!authHeader || !authHeader.startsWith('Bearer ')) {
    return res.status(401).json({ message: 'Accès non autorisé : Token manquant' });
  }

  const token = authHeader.split(' ')[1];
  try {
    const decoded = jwt.verify(token, process.env.JWT_SECRET!) as { id: string; role: string };
    req.user = decoded;
    next();
  } catch (error) {
    return res.status(403).json({ message: 'Token invalide ou expiré' });
  }
};

export const exigerRole = (...rolesAutorises: string[]) => {
  return (req: AuthRequest, res: Response, next: NextFunction) => {
    if (!req.user || !rolesAutorises.includes(req.user.role)) {
      return res.status(403).json({ message: 'Droits insuffisants pour effectuer cette action' });
    }
    next();
  };
};
```

Développement Frontend Web (React & TypeScript)

3.1 Service Client Axios avec Gestion Automatique des Tokens

Le client Axios intercepte toutes les requêtes pour y injecter le token d'authentification et gère la déconnexion automatique si la session expire :

```
import axios from 'axios';

const apiClient = axios.create({
  baseURL: 'https://api.votre-domaine.com/api',
  headers: { 'Content-Type': 'application/json' }
});

// Intercepteur de requête : Injection du Token
apiClient.interceptors.request.use((config) => {
  const token = localStorage.getItem('token');
  if (token) {
    config.headers.Authorization = `Bearer ${token}`;
  }
  return config;
});

// Intercepteur de réponse : Gestion des erreurs 401 (Session Expirée)
apiClient.interceptors.response.use(
  (response) => response,
  (error) => {
    if (error.response && error.response.status === 401) {
      localStorage.removeItem('token');
      window.location.href = '/login';
    }
    return Promise.reject(error);
  }
);
```

Développement Mobile Cross-Platform (React Native / Expo)

4.1 Navigation Mobile avec React Navigation 6

Sur Mobile, la navigation s'organise en empilement d'écrans (Stack) et en onglets inférieurs (Bottom Tabs) :

```
import React from 'react';
import { NavigationContainer } from '@react-navigation/native';
import { createNativeStackNavigator } from '@react-navigation/native-stack';
import { createBottomTabNavigator } from '@react-navigation/bottom-tabs';

import LoginScreen from '../screens/LoginScreen';
import HomeScreen from '../screens/HomeScreen';
import ProfileScreen from '../screens/ProfileScreen';

const Stack = createNativeStackNavigator();
const Tab = createBottomTabNavigator();

function MainTabs() {
  return (
    <Tab.Navigator>
      <Tab.Screen name="Accueil" component={HomeScreen} />
      <Tab.Screen name="Profil" component={ProfileScreen} />
    </Tab.Navigator>
  );
}

export default function AppNavigator() {
  return (
    <NavigationContainer>
      <Stack.Navigator initialRouteName="Login">
        <Stack.Screen name="Login" component={LoginScreen} options={{ headerShown: false }} />
        <Stack.Screen name="Main" component={MainTabs} options={{ headerShown: false }} />
      </Stack.Navigator>
    </NavigationContainer>
  );
}
```

):

Réalisé par Salomon — Développeur Full-Stack & Consultant IT

Édition Officielle 2026

4.2 Stockage Sécurisé des Tokens sur Mobile avec Expo SecureStore

Sur iOS et Android, `localStorage` n'existe pas. On utilise **Expo SecureStore** (utilisant le Keychain iOS et le Keystore Android) :

```
import * as SecureStore from 'expo-secure-store';

// Sauvegarder le token de manière chiffrée
export async function sauvegarderToken(token: string) {
  await SecureStore.setItemAsync('user_token', token);
}

// Récupérer le token chiffré
export async function recupererToken(): Promise<string | null> {
  return await SecureStore.getItemAsync('user_token');
}
```

Temps Réel (Socket.io), Docker & Déploiement Cloud

5.1 Serveur WebSockets Temps Réel avec Socket.io

Permet d'envoyer des notifications et messages instantanés en direct entre le serveur et tous les clients Web et Mobiles connectés :

```
import { Server } from 'socket.io';
import http from 'http';
import app from './app';

const server = http.createServer(app);
const io = new Server(server, {
  cors: { origin: '*' }
});

io.on('connection', (socket) => {
  console.log(`Client connecté : ${socket.id}`);

  socket.on('nouveau_message', (data) => {
    // Diffusion du message à TOUS les clients connectés
    io.emit('afficher_message', data);
  });

  socket.on('disconnect', () => {
    console.log('Client déconnecté');
  });
});

server.listen(5000, () => console.log('Serveur lancé sur le port 5000'));
```

6.1 Fichier Dockerfile pour Déploiement Cloud

Containerisez votre API Node.js pour garantir qu'elle s'exécute exactement de la même manière sur

Réalisé par Salomon — Développeur Full-Stack & Consultant IT

Édition Officielle 2026

```
# Image officielle Node.js légère
FROM node:18-alpine

# Répertoire de travail
WORKDIR /app

# Copie des fichiers de dépendances
COPY package*.json ./

# Installation des dépendances de production
RUN npm install --production

# Copie du code source
COPY . .

# Compilation TypeScript
RUN npm run build

# Port d'écoute
EXPOSE 5000

# Commande de démarrage
CMD ["node", "dist/app.js"]
```

6.2 Publication Mobile Android & iOS via Expo EAS Build

Commandes CLI officielles pour générer les fichiers binaires prêts pour le Google Play Store (`.aab`) et l'Apple App Store (`.ipa`) :

```
# 1. Installation du CLI officiel Expo EAS
npm install -g eas-cli

# 2. Connexion à votre compte Expo
eas login

# 3. Configuration du projet mobile
eas build:configure
```

Réalisé par Salomon — Développeur Full-Stack & Consultant IT

Édition Officielle 2026

FÉLICITATIONS !

Vous avez suivi l'intégralité du processus de création d'une solution numérique moderne de A à Z.
Vous maîtrisez désormais l'architecture Full-Stack complète, la sécurité, le temps réel et le déploiement multi-plateforme !

Guide de l'Autodidacte : Devenir Développeur Full-Stack Autonome

7.1 Feuille de Route d'Apprentissage Autonome (Roadmap Step-by-Step)

Devenir développeur Full-Stack sans l'aide d'un tiers exige une discipline rigoureuse et une stratégie en 5 étapes progressives :

Étape	Compétences Cibles	Projets Pratiques à Construire	Ressources Conseillées
Étape 1 (Mois 1-2) Bases Web	HTML5 sémantique, CSS3 (Flexbox, Grid, Responsive), JavaScript ES6+ (Promises, Async/Await, Array Methods).	Site portfolio personnel, Page E-commerce statique.	MDN Web Docs, freeCodeCamp, W3Schools.
Étape 2 (Mois 3-4) Frontend & TS	React.js, TypeScript, Hooks (useState, useEffect), React Router, Styling (TailwindCSS / CSS Modules).	Application Météo API, Dashboard de Gestion de Tâches (Todo List avancée).	Documentation officielle React, Académie TypeScript.
Étape 3 (Mois 5-6) Backend & BD	Node.js, Express.js, MongoDB (Mongoose), PostgreSQL, Authentification JWT, Hashage Bcrypt, API REST.	API REST de Blog avec système d'authentification et d'upload d'images.	Documentations Express & Mongoose.

Réalisé par Salomon — Développeur Full-Stack & Consultant IT				Édition Officielle 2026	
Étape 4 (Mois 7-8) Mobile Cross-Platform	React Native, Expo CLI, React Navigation, Expo SecureStore, Requêtes API sur Mobile.	Application Mobile de Commande de Repas ou de Messagerie Simple.	Documentation officielle Expo / React Native.		
Étape 5 (Mois 9-12) DevOps & Pro-Level	Git / GitHub Workflows, Docker, Linux Terminal, CI/CD, Déploiement Cloud (Render, Vercel, Railway).	Application SaaS Full-Stack (Web + Mobile + Backend + Base de Données en ligne).	Docker Docs, GitHub Actions Guide.		

7.2 Les 5 Erreurs Fatales à Éviter en Autodidacte

NE TOMBEZ PAS DANS CES PIÈGES :

- **1. Le piège du "Tutorial Hell"** : Regarder des centaines d'heures de cours vidéo sans rien coder par soi-même. *Solution : Règle du 20/80 (20% de théorie vidéo, 80% de pratique autonome).*
- **2. Zapper les Fondations JavaScript** : Sauter directement sur React sans comprendre comment fonctionnent les Promises, l'asynchronisme ou la portée des variables.
- **3. Apprendre trop de langages à la fois** : Essayer d'apprendre Python, PHP, Java, Flutter et React en même temps. *Masterisez d'abord la stack Full-JavaScript (React + Node.js + React Native).*
- **4. Ignorer Git et GitHub** : Coder des projets importants sur son ordinateur sans jamais faire de commits ni publier sur GitHub.
- **5. Abandonner face aux bugs** : Se décourager au premier message d'erreur. *Apprenez à lire les logs d'erreur de haut en bas et à chercher sur Google / Stack Overflow.*

Kit Commercial & Négociation Client : Gagner des Projets IT

8.1 Les 7 Questions & Réponses Clés en Entretien Client pour Décrocher des Contrats

Lors d'un entretien de négociation avec un client potentiel, le développeur ne doit pas parler comme un technicien mais comme un **Partenaire de Croissance Métier**. Voici les réponses types qui rassurent le client et vous font remporter le marché :

QUESTION CLIENT 1 : "POURQUOI DEVRAIS-JE VOUS CHOISIR VOUS PLUTÔT QU'UNE GROSSE AGENCE WEB ?"

Réponse Gagnante : « Avec une grande agence, votre projet sera dilué entre plusieurs stagiaires ou chefs de projets juniors. En travaillant directement avec moi, vous avez un interlocuteur unique, expert et ultra-réactif de A à Z. Je m'investis personnellement dans votre réussite avec des coûts d'infrastructure optimisés et zéro frais de structure inutile. »

QUESTION CLIENT 2 : "VOTRE DEVIS EST AU-DESSUS DE NOTRE BUDGET. POUVEZ-VOUS FAIRE UNE BAISSE DE PRIX ?"

Réponse Gagnante : « Je comprends parfaitement vos contraintes budgétaires. Cependant, réduire le prix sans toucher au périmètre dégraderait la qualité et la sécurité de votre application. Ce que je vous propose, c'est de réduire la portée de la Version 1 (MVP) en ne gardant que les 3 fonctionnalités vitales pour respecter votre budget actuel, puis de développer les options secondaires dans une Phase 2 une fois votre projet rentable. »

QUESTION CLIENT 3 : "COMBIEN DE TEMPS PRENDRA LA LIVRAISON DE L'APPLICATION ?"

Réponse Gagnante : « Pour garantir une qualité optimale, nous fonctionnons par livraison par

Réalisé par Salomon — Développeur Full-Stack & Consultant IT

Édition Officielle 2026

... sera livrée en [X] semaines conformément à l'échéancier des TDR. »

QUESTION CLIENT 4 : "QUE SE PASSE-T-IL SI UN BUG SURVIENT APRÈS LA LIVRAISON ET LE PAIEMENT FINAL ?"

Réponse Gagnante : « Tous mes contrats incluent une **Garantie Corrective Gratuite de 6 mois** post-livraison. Si un dysfonctionnement technique lié au code initial apparaît, je le corrige immédiatement à mes frais. De plus, nous pouvons mettre en place un contrat de maintenance préventive pour les évolutions futures. »

QUESTION CLIENT 5 : "À QUI APPARTIENT LE CODE SOURCE UNE FOIS LE PROJET ACHEVÉ ?"

Réponse Gagnante : « Dès le règlement du solde final, la totalité des droits de propriété intellectuelle et l'intégralité du code source vous sont transférés sur votre propre dépôt GitHub d'entreprise. Vous êtes 100% propriétaire de votre solution. »

8.2 Modèle de Proposition Technique et Financière (Devis Professionnel)

PROPOSITION TECHNIQUE ET FINANCIÈRE

Projet : Développement Application Web & Mobile "Nom Client"

Prestataire : Tchibkere Salomon (Développeur Full-Stack & Consultant)

Client : [Nom de l'Entreprise / Organisation Client]

Date : 30 Août 2026

1. PÉRIMÈTRE DES PRESTATIONS :

- Conception UI/UX & Maquettes Figma interactives
- API Backend sécurisée (Node.js/Express, JWT, Auth, MongoDB)
- Application Web Responsive (React + Vite)

2. ÉCHÉANCIER DE LIVRAISON ET DE PAIEMENT :

Réalisé par Salomon — Développeur Full-Stack & Consultant IT

Édition Officielle 2026

- Jalon 3 (30%) : Recette Finale, Déploiement & Solde —→ [Montant 3]

3. GARANTIES :

- 6 Mois de garantie corrective gratuite
 - Transfert 100% de la Propriété Intellectuelle du Code Source
-

Les Secrets d'Infiltré pour Percer & Exceller dans le Développement Software

9.1 Les 5 Secrets Technologiques de l'Ingénierie Logicielle Professionnelle

Les développeurs d'élite ne sont pas plus intelligents : ils appliquent des principes d'ingénierie ignorés du grand public :

- **1. Le Secret des "Design Patterns" (Patrons de Conception) :** Comprendre les architectures réutilisables (Singleton, Factory, Repository, MVC). Séparer hermétiquement la logique métier des requêtes de base de données.
- **2. Le Secret du Clean Code & de la Refactorisation :** Bannir les noms de variables vagues comme `x`, `temp` ou `data`. Écrire du code qui se lit comme une phrase en français/anglais.
- **3. Le Secret du Debugging Pro :** Ne plus mettre des `console.log` partout au hasard. Utiliser les **Breakpoints dans VS Code** et l'inspecteur **Chrome DevTools (Network & Performance Tab)** pour trouver l'origine exacte des bugs en 10 secondes.
- **4. Le Secret de l'Automatisation par l'IA :** Utiliser ChatGPT / Copilot / Claude non pas pour remplacer la réflexion, mais pour **générer les tests unitaires**, valider la syntaxe et accélérer le codage par 5.
- **5. Le Secret de la Gestion des Secrets (.env) :** Ne JAMAIS pousser de mots de passe, tokens ou clés API sur GitHub. Toujours ajouter `.env` dans `.gitignore` dès la création du projet.

9.2 Le Secret pour Obtenir des Clients & un Emploi sans Expérience préalable

Sans diplôme prestigieux ni années d'expérience officielle, la preuve par le code est l'arme absolue pour s'imposer :

- **Arrêter les Projets Scolaires Génériques** : Ne montrez **JAMAIS** une calculatrice ou une Todo

Réalisé par Salomon — Développeur Full-Stack & Consultant IT

Édition Officielle 2026

- **Construire 3 Projets "Monstres" Complets** :

1. Un SaaS Web complet (Ex: Application de gestion avec abonnement et paiements Stripe).
2. Une Application Mobile multi-plateforme avec notifications push et géolocalisation.
3. Un Tableau de Bord analytique en temps réel interfacé avec des WebSockets.

- **La Règle du README Impeccable** : Chaque dépôt GitHub doit posséder un titre clair, une vidéo GIF de démonstration, la liste des technologies et la commande pour lancer le projet en 1 clic.

9.3 Réussir les Entretiens Techniques & Négociation du Tarif (TJM)

En entretien technique, les recruteurs cherchent à tester votre méthodologie de résolution de problèmes :

Étape de l'Entretien	Ce que le Recruteur Évalue	La Stratégie d'Excellence
1. Culture & Architecture	Compréhension du Web, des APIs REST, du CORS et du cycle de vie des composants.	Parler avec un vocabulaire précis : <i>Asynchronisme, Immutabilité, Middleware, Injection de dépendances.</i>
2. Coding Live / Algo	Capacité à analyser un problème sous pression et à réfléchir à voix haute.	Toujours expliciter sa réflexion avant d'écrire la moindre ligne de code. Proposer d'abord une solution simple, puis l'optimiser.
3. Négociation Tarifaire (TJM)	Posture professionnelle et valeur perçue.	Ne jamais donner un chiffre au hasard. Ne jamais négocier à la baisse sans ajuster le périmètre de travail.

Psychologie de Vente & Conviction Express : Gagner la Confiance du Client en 5 Min

10.1 La Règle des 3 Premières Minutes (Écoute Active & Inversion de Posture)

Les 180 premières secondes d'une interaction commerciale déterminent si le client vous percevra comme un **Exécutant interchangeable** ou comme un **Expert Incontournable** :

L'ERREUR FATALE DES DÉVELOPPEURS DÉBUTANTS :

Parler immédiatement de code, de frameworks (React, Node.js, Python), d'algorithmes ou d'outils techniques. Le client chef d'entreprise ne cherche pas à acheter du code : **il cherche à résoudre un problème métier ou à augmenter son chiffre d'affaires.**

Les 3 Piliers de l'Inversion de Posture :

- 1. La Règle du Miroir & Reformulation Métier** : Laissez le client parler pendant les 2 premières minutes sans l'interrompre. Puis reformulez avec SES propres mots :
« Si je résume bien votre situation, votre enjeu principal n'est pas simplement d'avoir une application mobile de plus, mais de réduire de 50% le temps de gestion de vos dossiers et d'éliminer les erreurs manuelles d'encaissement. Est-ce exactement cela ? »
- 2. La Posture du Diagnosticien / Médecin IT** : Posez des questions d'investigation stratégiques :
 - « Pourquoi souhaitez-vous lancer ce projet maintenant plutôt que dans 6 mois ? »
 - « Quel est l'impact financier quotidien pour votre organisation si ce système n'est pas en ligne d'ici la fin du trimestre ? »
- 3. La Preuve Visuelle "Show, Don't Tell" en 30 secondes** : Ne perdez pas de temps en longs discours. Sortez votre smartphone ou votre PC et montrez une vidéo GIF ou une démonstration live

10.2 Le Script Psychologique à Utiliser pour Gagner la Confiance Immédiate

Réalisé par Salomon — Développeur Full-Stack & Consultant IT

Édition Officielle 2026

Pour éliminer définitivement la peur du risque chez le client et déclencher sa signature immédiate, déroulez ce script psychologique en 4 phrases :

Étape Psychologique	Script Mot-pour-Mot à Prononcer sur un Ton Calme et Assuré
Étape 1 : Valider le Diagnostic	<i>« Votre projet est extrêmement clair. C'est un cas de figure classique que j'ai déjà eu l'occasion d'architecturer avec succès sur d'autres projets d'envergure. »</i>
2. Feuille de Route Sans Jargon	<i>« Pour sécuriser votre temps et votre budget, nous allons procéder en 3 étapes simples : validation des maquettes Figma → prototype fonctionnel testable sur votre téléphone dans 14 jours → déploiement final. »</i>
3. Suppression du Risque Financier	<i>« Vous n'avez aucun risque à prendre : la prestation est découpée par jalons. Vous ne débloquez le deuxième virement que lorsque vous avez vous-même testé et validé la version Beta intermédiaire. »</i>
4. Garantie d'Excellence	<i>« De plus, une fois le projet en ligne, vous bénéficiez d'une garantie corrective gratuite de 6 mois pour corriger le moindre imprévu à mes frais. »</i>

10.3 Les 5 Signaux Comportementaux qui Déclenchent la Confiance Immédiate

- **1. La Franchise Brutale ("Radical Candor") :** Si le client propose une idée absurde ou trop coûteuse, dites-lui avec respect : *« En toute honnêteté, je vous déconseille de dépenser votre argent là-dessus pour la Version 1. Concentrons-nous sur ce qui va vous rapporter des clients immédiatement. »* → Cette honnêteté détruit 100% de la méfiance !
- **2. Des Supports Contractuels Irréprochables :** Présenter des documents TDR et un devis sous forme de PDF épuré au design exécutif (comme le présent manuel PDF).
- **3. La Réactivité Exemplaire :** Répondre aux messages avec une réponse structurée en puces dans un délai maîtrisé.
- **4. L'Assurance Vocale & le Regard Direct :** Parler posément, sans débit précipité, en articulant chaque choix d'architecture avec sérénité.

MANUEL MASTERCLASS — DÉVELOPPEMENT WEB & MOBILE DE A À Z

SALOMON TCHIBKERE

recapitulatif par mail ce soir à 18h → L'envoyer à 17h55).

Le Lexique & Glossaire Exhaustif du Développeur Full-Stack (Plus de 60 Termes)

11.1 Dictionnaire Web, Frontend, UI/UX & React

- **API (Application Programming Interface)** : Interface logicielle permettant à deux systèmes (comme le Frontend Web/Mobile et le serveur Backend) de communiquer de manière sécurisée.
- **SPA (Single Page Application)** : Application web dynamique (React/Vite) qui charge une seule page HTML et met à jour l'écran sans recharger la page complète.
- **PWA (Progressive Web App)** : Site web optimisé fonctionnant comme une application mobile native (installable sur l'écran d'accueil et disponible hors-ligne).
- **Virtual DOM** : Copie en mémoire du DOM HTML maintenue par React pour recalculer uniquement les éléments modifiés à l'écran (Algorithme de Diffing).
- **Responsive Design** : Conception d'interface adaptative s'ajustant automatiquement sur Smartphones, Tablettes et Ordinateurs.
- **State & Props** : L'état local modifiable d'un composant React (`useState`) vs les propriétés en lecture seule transmises par un composant parent.
- **Custom Hook** : Fonction personnalisée React (ex: `useAuth`) permettant d'encapsuler et de réutiliser de la logique métier dans toute l'application.

11.2 Dictionnaire Backend, Base de Données & Sécurité

- **API REST / RESTful** : Architecture d'API standardisée reposant sur les méthodes HTTP (`GET` , `POST` , `PUT` , `DELETE` , `PATCH`).
- **JWT (JSON Web Token)** : Jeton cryptographique compact et autonome transmis dans les en-têtes HTTP pour authentifier un utilisateur sans état serveur (Stateless).
- **CRUD** : Acronyme représentant les 4 opérations de base sur les données : **C**reate (Créer). **R**ead

-
- **ORM / ODM** : Outils (Mongoose, Prisma) traduisant les requêtes de base de données en objets
-

Réalisé par Salomon — Développeur Full-Stack & Consultant IT

Édition Officielle 2026

- **NoSQL vs SQL** : Base de données orientée documents flexibles (MongoDB) vs Base de données relationnelle structurée sous forme de tables liées (PostgreSQL).
- **Bcrypt** : Fonction de hachage cryptographique sécurisée à sens unique avec grain de sel (Salt) utilisée pour protéger les mots de passe.
- **CORS (Cross-Origin Resource Sharing)** : Mécanisme de sécurité HTTP restreignant les requêtes provenant de domaines non autorisés vers l'API.
- **Rate Limiting** : Limitation automatique du nombre de requêtes autorisées par IP pour contrer les attaques de force brute et le déni de service (DDoS).
- **RBAC (Role-Based Access Control)** : Contrôle d'accès par rôles (ex : `ADMIN` , `FORMATEUR` , `ETUDIANT`).

11.3 Dictionnaire Mobile & DevOps Cloud

- **React Native & Expo CLI** : Framework et boîte à outils permettant d'écrire des applications mobiles Android et iOS natives en JavaScript.
- **Bridge Natif** : Passerelle logicielle faisant communiquer le fil d'exécution JavaScript avec les APIs natives iOS/Android.
- **SecureStore / Keychain** : Espace de stockage hautement chiffré du système d'exploitation mobile réservé aux jetons JWT et données sensibles.
- **EAS (Expo Application Services)** : Infrastructure Cloud compilant automatiquement vos fichiers exécutables Android (`.apk` / `.aab`) et iOS (`.ipa`).
- **Docker & Conteneur** : Technologie d'empaquetage garantissant qu'une application s'exécute de manière identique sur n'importe quel serveur.
- **CI/CD (Continuous Integration / Continuous Deployment)** : Pipeline automatisé (GitHub Actions) testant et déployant le code en ligne à chaque commit.

11.4 Dictionnaire Business, Gestion de Projet Agile & Freelance

- **MVP (Minimum Viable Product)** : Version initiale simplifiée d'un produit contenant uniquement les fonctionnalités indispensables pour tester le marché.
- **Méthode Agile / Scrum** : Méthodologie de développement itérative organisée en cycles de 2

MANUEL MASTERCLASS — DÉVELOPPEMENT WEB & MOBILE DE A À Z

SALOMON TCHIBKERE

• **CdC & TDR** : *Cahier des Charges (spécifications techniques) & Termes de Référence (cadre*

Réalisé par Salomon — Développeur Full-Stack & Consultant IT

Édition Officielle 2026

- **TJM (Taux Journalier Moyen)** : Prix facturé par un développeur indépendant pour une journée de travail (ex: 200 000 FCFA / jour).
- **SLA (Service Level Agreement)** : Contrat de niveau de service garantissant le taux de disponibilité d'une application (ex: 99.9%).

Intégration des API de Paiement (Mobile Money & Stripe)

12.1 Flux de Paiement Mobile Money (Orange Money, MTN, Wave) & Webhooks

En Afrique et dans les marchés émergents, 80% des transactions numériques s'effectuent par Mobile Money via des passerelles comme **CinetPay**, **PaySika** ou **Bizao** :

```
import axios from 'axios';

// Initialiser une transaction Mobile Money (Orange Money / MTN)
export async function initierPaiementMobile(montant: number, telephone: string) {
  const payload = {
    apikey: process.env.CINETPAY_API_KEY,
    site_id: process.env.CINETPAY_SITE_ID,
    transaction_id: `TX_${Date.now()}`,
    amount: montant,
    currency: 'XAF',
    description: 'Paiement de la Formation Web & Mobile',
    notify_url: 'https://api.votre-domaine.com/webhook/cinetpay',
    customer_phone_number: telephone
  };

  const response = await axios.post('https://api-checkout.cinetpay.com/v2/payment', payload);
  return response.data;
}
```

12.2 Traitement des Webhooks de Confirmation de Paiement

Ne confirmez JAMAIS une commande côté client ! Le serveur marchand doit attendre la notification

Réalisé par Salomon — Développeur Full-Stack & Consultant IT

Édition Officielle 2026

```
export async function traiterWebhookPaielement(req: Request, res: Response) {
  const { transaction_id, status } = req.body;

  if (status === 'ACCEPTED') {
    // 1. Valider la commande en Base de Données
    await Commande.updateOne({ transactionId: transaction_id }, { status: 'PAYEE' });

    // 2. Envoyer le reçu par e-mail ou débloquent l'accès
    console.log(`Paielement réussi pour la transaction : ${transaction_id}`);
  }

  return res.status(200).json({ received: true });
}
```


Maintenance, Monitoring & Plan de Reprise d'Activité (PRA)

13.1 Gestion du Processus Backend avec PM2 & Monitoring Sentry

En production, si votre serveur Node.js plante à cause d'une erreur inattendue, **PM2** le relance automatiquement en moins de 100 millisecondes :

```
# 1. Installation globale de PM2
npm install -g pm2

# 2. Démarrage de l'API en mode Cluster (Haute Disponibilité multi-coeurs CPU)
pm2 start dist/app.js -i max --name "api-prod"

# 3. Sauvegarde de la configuration pour redémarrage automatique après reboot serveur
pm2 save
pm2 startup
```

13.2 Script de Sauvegarde Automatique de la Base de Données (Backup Cron)

Un développeur senior sauvegarde la base de données de ses clients chaque nuit à minuit via un script **Cron Job** :

```
#!/bin/bash
# Script de Backup PostgreSQL Automatique
DATE=$(date +%Y-%m-%d_%H-%M-%S)
BACKUP_DIR="/var/backups/postgres"
mkdir -p $BACKUP_DIR
```

```
# Suppression des sauvegardes de plus de 30 iours
```

Réalisé par Salomon — Développeur Full-Stack & Consultant IT

Édition Officielle 2026

```
echo "Sauvegarde réussie le $DATE"
```



Modèles de Contrats Juridiques & Accords de Confidentialité (NDA)

14.1 Modèle de Contrat de Prestation de Services Informatiques Prêt à Signer

CONTRAT DE PRESTATION DE SERVICES INFORMATIQUES

ENTRE LES SOUSSIGNÉS :

1. Le Prestataire : Tchibkere Salomon, Développeur Full-Stack & Consultant.
2. Le Client : [Nom de la Société / Client].

ARTICLE 1 : OBJET DU CONTRAT

Le Prestataire s'engage à réaliser la conception, le développement Web/Mobile et le déploiement de la solution logicielle décrite dans le Cahier des Charges.

ARTICLE 2 : PROPRIÉTÉ INTELLECTUELLE

La totalité des droits d'auteur et du code source est transférée au Client dès le paiement intégral du solde final prévu aux Termes de Référence (TDR).

ARTICLE 3 : GARANTIE CORRECTIVE

Le Prestataire offre une garantie corrective gratuite d'une durée de 6 (six) mois à compter de la date de recette finale.

Fait à Yaoundé, le [Date]

Le Prestataire (Signature)

Le Client (Signature)

14.2 Accord de Non-Divulcation & Confidentialité (NDA - Non-Disclosure Agreement)

Le Prestataire s'engage à conserver la confidentialité la plus stricte concernant

Réalisé par Salomon — Développeur Full-Stack & Consultant IT

Édition Officielle 2026

Cet engagement reste valable pour une durée de 3 (trois) ans après la fin du contrat.

